

معرفی مسئله

در کلاس‌های حضوری یا جلسات آموزشی، ضبط صدا و داشتن متن قابل جستجو از محتوای جلسه می‌تواند برای مرور درس، غیبت دانشجویها، و تهیه خلاصه کاربردی باشد. هدف این پروژه ساخت یک سامانه «ضبط کلاس» است که همزمان:

- صدای کلاس را به صورت پایدار ضبط کند،
- فایل‌های صوتی را در سرور ذخیره و در یک پنل تحت وب نمایش دهد،
- بعد از ضبط، روی صدا کاهش نویز انجام دهد و گفتار را به متن فارسی تبدیل کند،
- از متن تولیدشده، خلاصه‌ی قابل استفاده بسازد،
- و در کنار این‌ها، وضعیت محیط (دما و رطوبت) را روی نمایشگر دستگاه نشان دهد.

طراحی سیستم

معماری سیستم به صورت کلی از سه بخش تشکیل شده است:

1. بخش سخت‌افزار (دستگاه ضبط)

یک برد ESP32 به عنوان هسته اصلی استفاده شده که به سنسور DHT11 (برای دما/رطوبت)، نمایشگر TFT مدل ST7735 (برای نمایش اطلاعات)، و میکروفون دیجیتال INMP441 (برای دریافت صوت با I2S) متصل است. دستگاه از طریق WiFi به شبکه وصل می‌شود.

2. بخش ارتباطی (انتقال زنده صوت و کنترل)

برای ارتباط زنده بین دستگاه و سرور، دو مسیر WebSocket در نظر گرفته شده است:

- مسیر صوت برای ارسال بسته‌های صوتی به صورت باینری،
- مسیر کنترل/دستگاه برای دریافت فرمان‌های START/STOP و ارسال وضعیت ضبط.

3. بخش سرور و پردازش (پنل وب + تبدیل گفتار به متن)

روی سرور، یک برنامه Flask برای صفحات وب (دانشجو/استاد) اجرا می‌شود و همزمان یک WebSocket Server برای دریافت صوت و کنترل دستگاه فعال است. پس از ضبط، اسکریپت پردازش (speech.py) اجرا می‌شود تا:

- فایل پاک‌سازی‌شده (کاهش نویز) تولید کند،
- متن گفتار را استخراج و اصلاح کند،
- و خلاصه بسازد؛ سپس همه خروجی‌ها در پنل نمایش داده می‌شوند.

جزئیات پیاده‌سازی

(1) مونتاژ و اتصالات سخت‌افزاری

- تغذیه همه ماژول‌ها از 3.3V و GND مشترک انجام شده است.
- اتصال DHT11: پایه DATA به GPIO27 و یک مقاومت Pull-up حدود 10kΩ بین DATA و 3.3V قرار داده شده است.
- اتصال (SPI) TFT ST7735: پایه‌های SCK=GPIO18، MOSI=GPIO23، DC=GPIO21، RST=GPIO4، CS=GPIO5 و تغذیه 3.3V.
- اتصال (I2S) INMP441: پایه SCK=GPIO26، WS=GPIO25، SD=GPIO32 و انتخاب کانال Left با اتصال L/R به GND.

(2) برنامه ESP32 (فایل STM32.ino)

- خواندن سنسور DHT11 هر ۲ ثانیه انجام می‌شود و مقدار دما و رطوبت روی TFT چاپ می‌گردد.
- میکروفون با I2S در نرخ نمونه‌برداری 16000Hz تنظیم شده و داده‌ی 32 بیت دریافت می‌شود، سپس به PCM 16-bit تبدیل می‌گردد.
- ارسال صوت به صورت تکه‌های کوتاه و باینری روی WebSocket صوت انجام می‌شود تا قطع و وصل و تأخیر کمتر شود.
- دستگاه دو اتصال WebSocket برقرار می‌کند:
 - یکی برای صوت (/audio)
 - یکی برای کنترل (/device)
- با دریافت فرمان START ضبط فعال می‌شود و ارسال صوت شروع می‌گردد؛ با STOP ضبط متوقف می‌شود.
- مکانیزم reconnect در کد وجود دارد تا اگر ارتباط قطع شد، دستگاه دوباره تلاش کند وصل شود.
- حلقه اصلی طوری نوشته شده که ابتدا بخش صوت (loop_audio) اجرا شود و سپس نمایش (loop_dht) DHT تا احتمال از دست رفتن داده صوتی کمتر شود.

(3) سرور و پنل وب (فایل server.py + قالب‌ها/استاتیک)

- سرور HTTP با Flask روی پورت 8080 اجرا می‌شود و نقش‌ها از صفحه اصلی انتخاب می‌شوند (دانشجو/استاد).
- ورود استاد با رمز ثابت انجام می‌شود و وضعیت ورود با session نگهداری می‌شود.
- بخش WebSocket روی پورت 8765 فعال است و بر اساس مسیر درخواست:
 - audio/: دریافت باینری صوت و ذخیره در فایل موقت PCM
 - device/: دریافت پیام‌های وضعیت و ارسال فرمان‌های START/STOP
- ذخیره‌سازی صوت:
 - هنگام START یک فایل PCM در tmp_pcm ساخته می‌شود،
 - هنگام STOP فایل PCM به WAV تبدیل و در recordings/ ذخیره می‌شود (mono, 16kHz, 16-bit).
- پنل دانشجو:
 - لیست فایل‌های خام (recordings/*.wav) و فایل‌های پاک‌سازی‌شده (recordings/clean/*.wav) را نشان می‌دهد،
 - قابلیت جستجو و مرتب‌سازی دارد،
 - برای فایل‌های clean دکمه «متن» و «خلاصه» دارد که از مسیرهای API متن را خوانده و در یک Modal نمایش می‌دهد.
- پنل استاد:
 - وضعیت اتصال دستگاه و وضعیت ضبط را نشان می‌دهد،
 - با دکمه‌های شروع/توقف از API فرمان می‌فرستد،
 - به‌روزرسانی وضعیت در کد فعلی با درخواست دوره‌ای به api/status/ انجام می‌شود.

4) تبدیل گفتار به متن و خلاصه‌سازی (فایل speech.py)

- همه فایل‌های WAV داخل recordings/ بررسی می‌شوند و اگر خروجی‌های متنی قبلاً ساخته نشده باشند پردازش انجام می‌گیرد.
- ابتدا کاهش نویز با noisereduce انجام و خروجی در recordings/clean/ ذخیره می‌شود.

- سپس مدل **Whisper (large-v3)** برای تبدیل گفتار فارسی به متن استفاده می‌شود.
- متن خام به یک مدل زبانی (از طریق **LangChain**) داده می‌شود تا:
 - خطاهای رایج **ASR** اصلاح شود و متن روان‌تر شود (بدون اضافه کردن اطلاعات جدید)،
 - و بعد خلاصه با ساختار مشخص تولید گردد.
- خروجی‌ها:
 - متن نهایی در **/recordings/texts/speech**
 - خلاصه در **/recordings/texts/summery**

نتایج

- سیستم توانسته فایل‌های ضبط‌شده را با نام‌گذاری زمانی تولید و در مسیر **/recordings** ذخیره کند (نمونه‌هایی مثل **wav.103515_20260203** و ... در پروژه موجود است).
- پس از اجرای پردازش، برای هر ضبط:
 - نسخه کاهش‌نویز داده‌شده در **/recordings/clean** ایجاد شده،
 - متن کامل فارسی و خلاصه‌ی ساختاریافته در پوشه‌های **texts/speech** و **texts/summery** ذخیره شده‌اند.
- در پنل دانشجو، فایل‌ها قابل پخش و دانلود هستند و متن/خلاصه بدون نیاز به خروج از صفحه در پنجره جدا نمایش داده می‌شود.
- در سمت دستگاه، نمایش دما و رطوبت روی **TFT** به صورت دوره‌ای انجام شده و هم‌زمان ارسال صوت با فرمان‌های **START/STOP** کنترل شده است.
- وجود **reconnect** و اجرای اولویت‌دار حلقه صوت کمک کرده است که ارسال صوت پایدارتر باشد و احتمال از دست رفتن داده کمتر شود.

جمع‌بندی

این پروژه یک نمونه عملی از ترکیب سخت‌افزار و پردازش گفتار است که در آن **ESP32** هم‌زمان وظیفه نمایش دما/رطوبت و ارسال زنده صوت را انجام می‌دهد و سرور نیز علاوه بر ذخیره‌سازی فایل‌های صوتی، یک پنل تحت وب برای استاد و دانشجو فراهم می‌کند. بخش پردازش (**speech.py**) با کاهش نویز، تبدیل گفتار به متن فارسی و تولید خلاصه، خروجی پروژه را از یک «ضبط ساده» به یک «محتوای قابل جستجو و قابل مرور» تبدیل کرده است. در مجموع، سیستم از

نظر عملکرد اصلی (ضبط، ذخیره، نمایش در پنل، تولید متن و خلاصه) کامل است و می‌تواند پایه‌ای برای توسعه‌های بعدی مثل امنیت بهتر، تنظیمات پویا، یا اجرای خودکار پردازش بعد از هر ضبط باشد.